

Database Systems I / Information Systems I

Relational Algebra — Complementary Material

Christian Rauch

(changed: 2026-05-03)

Selection σ

The **selection** operator filters rows that satisfy a condition.

Notation

$\sigma_{\text{condition}}(R)$

- ▶ Returns all tuples of R for which the condition holds.
- ▶ Condition may use $=$, $\neq$, $<$, $>$, $\leq$, $\geq$, combined with $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT).
- ▶ The result has the *same schema* as R .

Example: All artifacts with value over 10 000:

$$\sigma_{\text{value} > 10000}(\text{Artifacts})$$

Example: Sculptures made of bronze:

$$\sigma_{\text{material} = \text{'bronze'}}(\text{ArtifactSculptures})$$

Selection – Compound Conditions

Multiple predicates can be combined.

- ▶ **AND** ($\wedge$): both conditions must hold
- ▶ **OR** ($\vee$): at least one condition must hold
- ▶ **NOT** ($\neg$): condition must not hold

Example: Artifacts worth more than 5 000 created after 1900:

$$\sigma_{\text{value} > 5000 \wedge \text{year} > 1900}(\textit{Artifacts})$$

Example: Artists born before 1800 or after 1950:

$$\sigma_{\text{birthDate} < '1800-01-01' \vee \text{birthDate} > '1950-01-01'}(\textit{Artists})$$

Selection σ – In SQL

Selection maps directly to the WHERE clause.

```
1  -- sigma_{value > 10000}(Artifacts)
2  SELECT * FROM Artifacts WHERE value > 10000;
3
4  -- sigma_{material = 'bronze'}(ArtifactSculptures)
5  SELECT * FROM ArtifactSculptures WHERE material = 'bronze';
6
7  -- sigma_{value>5000 AND year>1900}(Artifacts)
8  SELECT * FROM Artifacts WHERE value > 5000 AND year > 1900;
9
10 -- sigma_{birthDate<'1800' OR birthDate>'1950'}(Artists)
11 SELECT * FROM Artists
12 WHERE birthDate < '1800-01-01' OR birthDate > '1950-01-01';
```

Projection π

The **projection** operator selects a subset of columns.

Notation

$$\pi_{A_1, A_2, \dots, A_k}(R)$$

- ▶ Returns only the listed attributes.
- ▶ Duplicate tuples are *eliminated* (set semantics).
- ▶ The result schema contains only $A_1, \dots, A_k$.

Example: Names of all artists:

$$\pi_{\text{name}}(\textit{Artists})$$

Example: Title and value of every artifact:

$$\pi_{\text{title, value}}(\textit{Artifacts})$$

Combining Selection and Projection

Selection and projection are often chained. Selection is applied *before* projection.

Example: Names of artists born after 1900:

$$\pi_{\text{name}}(\sigma_{\text{birthDate} > '1900-01-01'}(\textit{Artists}))$$

Example: Titles and values of oil paintings:

$$\pi_{\text{title, value}}(\textit{Artifacts} \bowtie \sigma_{\text{canvas} = 'oil'}(\textit{ArtifactPaintings}))$$

Projection π – In SQL

Projection maps to the SELECT column list. RA eliminates duplicates; SQL requires DISTINCT for the same effect.

```
1  --  $\pi_{\{name\}}(Artists)$ 
2  SELECT DISTINCT name FROM Artists;
3
4  --  $\pi_{\{title, value\}}(Artifacts)$ 
5  SELECT DISTINCT title, value FROM Artifacts;
6
7  --  $\pi_{\{name\}}(\sigma_{\{birthDate > '1900-01-01'\}}(Artists))$ 
8  SELECT DISTINCT name FROM Artists WHERE birthDate > '
   1900-01-01';
9
10 --  $\pi_{\{title, value\}}(Artifacts \Join \sigma_{\{canvas='oil'\}}(ArtifactPaintings))$ 
11 SELECT DISTINCT a.title, a.value
12 FROM   Artifacts a
13 JOIN   ArtifactPaintings ap ON a.id = ap.id
14 WHERE  ap.canvas = 'oil';
```

Union, Intersection, Difference

The three classical set operations require **union-compatible** operands (same number of attributes, compatible domains).

Operation	Symbol	Returns
Union	$R \cup S$	tuples in R or S (or both)
Intersection	$R \cap S$	tuples in both R and S
Difference	$R \setminus S$	tuples in R but not in S

Example: IDs of artifacts that are either paintings or sculptures:

$$\pi_{\text{id}}(\textit{ArtifactPaintings}) \cup \pi_{\text{id}}(\textit{ArtifactSculptures})$$

Set Operations – Examples

Intersection: IDs of artifacts that are *both* a painting and a sculpture (data quality check – should be empty):

$$\pi_{id}(ArtifactPaintings) \cap \pi_{id}(ArtifactSculptures)$$

Difference: Artists who influenced others but were never influenced themselves:

$$Influencers := \rho_{influencerId \rightarrow id}(\pi_{influencerId}(InfluencedBy))$$

$$Influencees := \rho_{influenceeId \rightarrow id}(\pi_{influenceeId}(InfluencedBy))$$

$$Answer := \pi_{name}(Artists \bowtie (Influencers \setminus Influencees))$$

Set Operations – In SQL

SQL provides UNION, INTERSECT, and EXCEPT directly. All three eliminate duplicates by default (like RA set semantics).

```
1  -- pi_{id}(ArtifactPaintings) UNION pi_{id}(
   ArtifactSculptures)
2  SELECT id FROM ArtifactPaintings
3  UNION
4  SELECT id FROM ArtifactSculptures;
5
6  -- Intersection (data quality check)
7  SELECT id FROM ArtifactPaintings
8  INTERSECT
9  SELECT id FROM ArtifactSculptures;
10
11 -- Difference: influenced others but never influenced
   themselves
12 SELECT influencerId AS id FROM InfluencedBy
13 EXCEPT
14 SELECT influenceeId      FROM InfluencedBy;
```

Rename ρ

The **rename** operator gives a relation or its attributes new names.

Notation

$\rho_{B_1 \rightarrow A_1, B_2 \rightarrow A_2}(R)$ renames attributes

$\rho_S(R)$ renames the relation itself to S

- ▶ Used to resolve **name conflicts** before a join.
- ▶ Used for **self-joins** (joining a table with itself).
- ▶ Used to make intermediate results readable via assignment notation.

Example: Rename `id` in *Artifacts* to `artifactId` so it can join with *ExhibitedAt*:

$$\rho_{id \rightarrow artifactId}(Artifacts)$$

Self-Join via Rename

To join a table with itself, create two copies with different attribute names.

Example: Pairs of artists (influencer name, influencee name):

$$A_1 := \rho_{id \rightarrow id_1, name \rightarrow name_1}(Artists)$$

$$A_2 := \rho_{id \rightarrow id_2, name \rightarrow name_2}(Artists)$$

$$Answer := \pi_{name_1, name_2} \left(A_1 \bowtie_{id_1=influencerId} InfluencedBy \right. \\ \left. \bowtie_{influenceeId=id_2} A_2 \right)$$

Rename ρ – In SQL

Renaming maps to table and column **aliases** (AS). Self-joins use two aliases of the same table.

```
1  -- rho_{id -> artifactId}(Artifacts): column alias
2  SELECT id AS artifactId FROM Artifacts;
3
4  -- Rename in a join context (alias resolves the name
   conflict)
5  SELECT ea.exhibitionTitle
6  FROM    ExhibitedAt ea
7  JOIN    Artifacts a ON ea.artifactId = a.id;
8
9  -- Self-join: pairs of artists (influencer, influencee)
10 SELECT ar1.name AS influencer,
11        ar2.name AS influencee
12 FROM    Artists ar1
13 JOIN    InfluencedBy ib ON ar1.id = ib.influencerId
14 JOIN    Artists ar2 ON ib.influenceeId = ar2.id;
```

Cartesian Product and Theta-Join

Cartesian Product $R \times S$

Combines every tuple of R with every tuple of S .

Result size: $|R| \cdot |S|$ tuples.

Theta-Join $R \bowtie_{\theta} S$

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

Keeps only those combined tuples satisfying condition θ .

Example: Artifacts together with their artist's name:

$$Artifacts \bowtie_{\text{artistId=id}} Artists$$

Example: Artifacts with value higher than another artifact by the same artist (self-join):

$$\pi_{\text{id}_1, \text{id}_2} \left(\rho_{\text{id} \rightarrow \text{id}_1, \text{value} \rightarrow v_1, \text{artistId} \rightarrow a_1} (Artifacts) \right. \\ \left. \bowtie_{a_1=a_2 \wedge v_1 > v_2} \rho_{\text{id} \rightarrow \text{id}_2, \text{value} \rightarrow v_2, \text{artistId} \rightarrow a_2} (Artifacts) \right)$$

Natural Join $\bowtie$

Natural Join $R \bowtie S$

Joins on *all common attribute names* and removes duplicate columns.

Equivalent to: $\pi_{\dots}(\sigma_{\text{common attrs equal}}(R \times S))$.

- ▶ Convenient when attribute names are aligned by design.
- ▶ Use **rename** (ρ) first when names don't match.

Example: Exhibitions an artifact was shown at (*ExhibitedAt* shares artifactId with renamed *Artifacts*):

$$ExhibitedAt \bowtie \rho_{id \rightarrow artifactId}(Artifacts)$$

Chaining Multiple Joins

Joins can be chained to traverse several relations. Use intermediate variable assignment for clarity.

Example: Names of artists and the exhibition titles of their exhibited works:

$$\begin{aligned} AE &:= \text{Artifacts} \bowtie_{\text{id}=\text{artifactId}} \text{ExhibitedAt} \\ \text{Answer} &:= \pi_{\text{name}, \text{exhibitionTitle}} (\text{Artists} \bowtie_{\text{id}=\text{artistId}} AE) \end{aligned}$$

Example: Media that advertised exhibitions where bronze sculptures were shown:

$$\begin{aligned} B &:= \rho_{\text{id} \rightarrow \text{artifactId}} (\sigma_{\text{material}=\text{'bronze'}} (\text{ArtifactSculptures})) \\ E &:= \pi_{\text{exhibitionTitle}} (\text{ExhibitedAt} \bowtie B) \\ \text{Answer} &:= \pi_{\text{name}} (\text{Media} \bowtie_{\text{name}=\text{mediumName}} (\text{Advertisements} \bowtie E)) \end{aligned}$$

Joins – In SQL

Every RA join corresponds to a JOIN ... ON in SQL. Chained joins translate to multiple JOIN clauses.

```
1  -- Artifacts JOIN_{artistId=id} Artists
2  SELECT a.title, ar.name
3  FROM   Artifacts a JOIN Artists ar ON a.artistId = ar.id;
4
5  -- Chained: artist names with their exhibition titles
6  SELECT ar.name, ea.exhibitionTitle
7  FROM   Artists ar
8  JOIN   Artifacts a ON ar.id = a.artistId
9  JOIN   ExhibitedAt ea ON a.id = ea.artifactId;
10
11 -- Media advertising bronze-sculpture exhibitions
12 SELECT DISTINCT m.name
13 FROM   Media m
14 JOIN   Advertisements ad ON m.name = ad.mediumName
15 JOIN   ExhibitedAt ea ON ad.exhibitionTitle = ea.
      exhibitionTitle
16 JOIN   ArtifactSculptures s ON ea.artifactId = s.id
17 WHERE  s.material = 'bronze';
```

Linear Notation and Variable Assignment

Long expressions are easier to read when broken into named sub-expressions.

Assignment

$Name(schema) := expression$

- ▶ The schema annotation is optional but clarifies attribute names.
- ▶ Sub-expressions can be reused later by name.
- ▶ The final expression is often called *Answer*.

Example: Newspapers that advertised an exhibition with oil paintings:

$$P' := \rho_{id \rightarrow artifactId} (\sigma_{canvas='oil'} (ArtifactPaintings))$$

$$P := \pi_{exhibitionTitle} (ExhibitedAt \bowtie P')$$

$$N := \sigma_{type='newspaper'} (Media \bowtie_{name=mediumName} Advertisements)$$

$$Answer := \pi_{name} (N \bowtie P)$$

Linear Notation – In SQL (CTEs)

Variable assignment in RA corresponds to **Common Table Expressions** (WITH clauses) in SQL.

```
1  -- Newspapers advertising exhibitions with oil paintings
2  WITH P AS (
3      -- P(exhibitionTitle): exhibitions with an oil painting
4      SELECT DISTINCT ea.exhibitionTitle
5      FROM    ExhibitedAt ea
6      JOIN    ArtifactPaintings ap ON ea.artifactId = ap.id
7      WHERE   ap.canvas = 'oil'
8  ),
9  N AS (
10     -- N: newspaper-advertisement pairs
11     SELECT m.name, ad.exhibitionTitle
12     FROM    Media m
13     JOIN    Advertisements ad ON m.name = ad.mediumName
14     WHERE   m.type = 'newspaper'
15  )
16  SELECT DISTINCT N.name
17  FROM    N JOIN P ON N.exhibitionTitle = P.exhibitionTitle;
```

Query Trees

A **query tree** is a tree representation of a relational algebra expression.

- ▶ **Leaves** are base relations.
- ▶ **Internal nodes** are operators (σ , π , $\bowtie$, ρ , ...).
- ▶ Execution proceeds **bottom-up**: leaves are read first, results flow up to the root.
- ▶ The root node produces the final result.

Example expression:

$$\pi_{\text{name}}(\sigma_{\text{value} > 10000}(\textit{Artifacts}) \bowtie_{\text{artistId} = \text{id}} \textit{Artists})$$

The tree has π at the root, $\bowtie$ below it, and $\sigma(\textit{Artifacts})$ and *Artists* as the two leaves.

Query Optimization

1. **Push selections down** as far as possible – filter rows early to reduce intermediate result sizes.
2. **Push projections down** – discard unneeded columns before joins.
3. **Perform the most restrictive selection first** – smallest intermediate results reduce the cost of subsequent operators.
4. **Avoid Cartesian products** – replace $R \times S$ followed by σ with a join $R \bowtie_{\theta} S$.
5. **Combine cascaded selections** or **split** them when pushing parts to different branches.